

VCS SystemVerilog Testbench Training Exercises

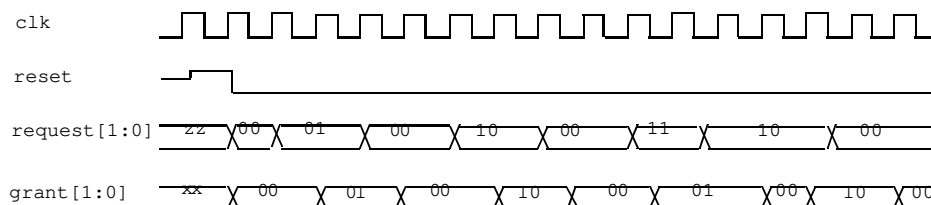
LAB 1: Getting Started with SV Testbench

Goal	Write a simple testbench for a 2-port arbiter, get familiar with interfaces, clocking blocks, program blocks, driving and sampling
Location	lab1
Design	Arbiter
Allocated Time	30 minutes

Relevant Files in lab1 directory:

hdl/arb.v	Arbiter DUT
hdl/arb_if.v	Arbiter interface
hdl/top.v	Top level netlist program
tests/test.v	Testbench

Sample Timing Diagram for Arbiter:



- 1) Complete the interface definition including the clocking block in **hdl/arb_if.v**. See **arb.v** for signal descriptions.
- 2) Complete the task **reset_test** in the file **tests/test.v**. Assert the reset signal (active high), make sure you are not making any requests for two cycles, and then check that the grant is all zeroes one cycle later.
- 3) Complete the task **request_grant_test** in **tests/test.v**. You should check that arbiter works correct for a single request on both channels, and for multiple requests. The arbiter implements a round-robin arbitration algorithm. Use in-line assertions as shown in **test.v**

** Search for “**LAB**” in the lab files to see where to add your code.

Solution in **lab1/.solution**

LAB 2: OOP Basics

Goal	Write a more structured testbench and get familiar with classes, randomization, threads, mailboxes
Location	lab2
Design	APB Interface
Allocated Time	60 minutes

Relevant Files in lab2 directory:

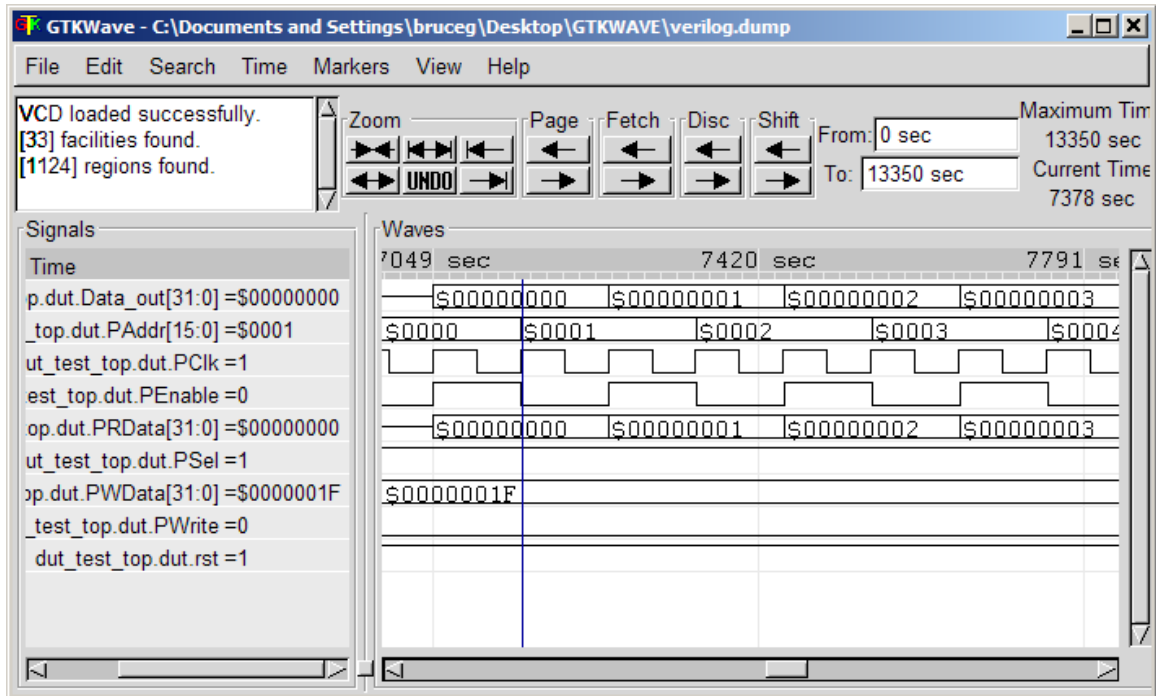
hdl/root.v	Global declarations & timescale
hdl/mem.v	APB interface DUT
hdl/top.v	Top level netlist
tests/test.v	Top level testbench program
apb_env/apb_if.sv	Interface / clocking block definition
apb_env/apb_trans.sv	Transactor class
apb_env/apb_gen.sv	Generator class
apb_env/apb_master.sv	Bus master class

Lab instructions:

- 1) Complete the transactor class **apb_trans** in **apb_env/apb_trans.sv**. Create 3 random variables, create method “**display**” to print out the random data, and create a method **copy** that returns a copy of the class data.
- 2) In **apb_env/apb_gen.sv**, complete the generator class **apb_gen**. Write the constructor for this class that initializes the mailbox, initializes the transactor object, and initializes the max number of transactions. Create the task **main** with a loop that sends **max_trans_cnt** random transactions to mailbox.
- 4) In **apb_env/apb_master.sv**, edit the constructor function **new**, and initialize the mailbox and create an APB transaction **tr**. In the **main** task, create an infinite loop to get transactions from the mailbox, decode them, and execute them with the **read**, **write**, and **idle** tasks.
- 5) Optional: Complete the main program in **tests/test.v**. Initialize the mailbox, call the constructor for the generator and the master, call the **main** & **reset** method of the master object, and call the **main** method of the generator object. Lastly, end the test when the generator ends, the mailbox empties, and then a few cycles more.

Timing Diagram for the APB interface:

Read Transaction: **PSEL** goes high at the rising edge of **PEnable** and **PRData** gets the value in address **PAddr**.



Write Transaction: **PSEL** should be high, **PWrite** should be high, and at the rising edge of **PEnable** and **PWData** is written to address at **PAddr**.

LAB 3: Virtual interfaces, monitors & scoreboards

Goal	Take lab 2 and add virtual interfaces, monitors & scoreboards
Location	lab3
Design	APB Interface
Allocated Time	60 minutes

Relevant Files in lab3 directory:

hdl/root.v	Global declarations & timescale
hdl/mem.v	APB interface DUT
hdl/top.v	Top level netlist
tests/test.v	Top level testbench program
apb_env/apb_defines.v	Address/data width parameters
apb_env/apb_if.sv	Interface / clocking block definition
apb_env/apb_trans.sv	Transactor class
apb_env/apb_gen.sv	Generator class
apb_env/apb_master.sv	
apb_env/apb_monitor.sv	Monitor class
env/env.sv	Environment class
env/scoreboard.sv	Scoreboard Class

Lab instructions:

- 1) In **apb_env/apb_gen.sv**, add the virtual function **end_of_test** that compares the number of transactions sent to the limit,.
- 2) Create virtual interface in the **apb_master** class, initialize it in the constructor. Hint: First review the interface and Master modports as defined in **apb_if**.
- 3) Create virtual interface in the **apb_monitor** class, initialize it in the constructor. In main, monitor the interface for transactions. Hint: First review the interface and Monitor modports as defined in **apb_if**.
- 4) Optional: In **env/scoreboard.sv**, create the main loop that waits on event **tr_mon_ready** and then does appropriate action: if write transaction, then store data in **reg_file**, if read transaction, then check for correct data, decrement **max_trans_cnt**, and if no more tests, then triggers event **ended**.

LAB 4: Constrained, Directed & Coverage Driven Testing

Goal	Build upon Lab3 by adding different type of tests using inheritance, constraint block, and covergroups
Location	lab4
Design	APB Interface
Allocated Time	60 minutes

Relevant Files in lab4 directory:

tests/test_directed.sv	Test with directed test by overriding random call
tests/test_constrained.sv	Test with directed test via constraint block
tests/test_cvr_driven.sv	Random test with coverage feedback

Lab instructions:

- 1) In **tests/test_01_directed.sv**, extend the **apb_gen** class, and provide a new implementation of the **get_transactions** task that creates 32 write transactions then 32 reads. See the next page for an OOP example. (Remember that the generator contains the variable **trans_cnt** that has the current transaction number.) Run the test with “**make 01**”.
- 2) In **tests/test_02_constrained.sv**, extend the **apb_gen** class, and create constraint block that sends 32 write transactions, and then 32 read transactions. Use a constraint block with the **if** constraint. Run with “**make 02**”.
- 3) In **tests/test_03_cvr_driven.sv**, create a functional covergroup with the following cover points:
tran: cover transaction type
addr : cover address
data : cover data with 4 bins, data {8'h00,8'h55,8'haa,8'hff}
 Create cross coverage of all three cover groups. Run with “**make 03**”.

```

// Original generator
class apb_gen;
int trans_cnt;

    function apb_trans get_transaction();
    // Create a new transaction
    endfunction
endclass

// Original environment
class env;
    apb_gen gen;
    task new;        // Construct environment
        gen = new;   // Create basic generator
    endtask

    task main;
        gen.main();  // Run the generator referenced by "gen"
    endtask
endclass : env

// New generator that has replacement task get_transaction
class my_gen extends apb_gen;
    function apb_tran get_transaction();
    if (trans_cnt > 32)
    // do something interesting
    endfunction
endclass : my_gen

// OOP code to replace apb_gen with my_gen
program test;
    env the_env;        // The basic environment
    my_gen genx;         // The extended generator

initial begin
    the_env = new;       // Create basic env and generator
    genx = new();        // Create extended generator
    the_env.gen = genx;  // Make env use extended generator

    the_end.run();       // Run test with extended generator
    $finish;
end
endprogram

```